

Giáo Án

Phiên bản web: <https://www.notion.so/Gi-o-n-1d7b9e967cbc80dfa434df27f2245531?pvs=4>

Giới thiệu và Thiết lập Môi trường

1. Mục tiêu

- Hiểu tại sao cần API Backend cho ứng dụng Frontend (React/React Native).
- Giới thiệu về NodeJS và ExpressJS.
- Khởi tạo thành công một dự án NodeJS và cài đặt ExpressJS.
- Chạy được một server Express đơn giản.

2. Nội dung

2.1 Tại sao cần Backend?

- Giải thích mô hình Client-Server.
- Vai trò của Backend: xử lý logic nghiệp vụ, quản lý dữ liệu, xác thực, cung cấp dữ liệu cho Frontend.
- Liên hệ với React/React Native: Frontend gọi API để lấy/gửi dữ liệu.

2.2 Giới thiệu NodeJS

- Là môi trường chạy JavaScript phía server.
- Ưu điểm: Non-blocking I/O, sử dụng chung ngôn ngữ JS với frontend.

2.3 Giới thiệu ExpressJS

- Là một framework web phổ biến cho NodeJS.
- Giúp xây dựng API và ứng dụng web nhanh chóng, dễ dàng.
- Cung cấp các tính năng như routing, middleware.

2.4 Thực hành: Thiết lập dự án

- Kiểm tra cài đặt NodeJS và npm/yarn: `node -v` , `npm -v` .
- Tạo thư mục dự án: `mkdir basic-express-api && cd basic-express-api`
- Khởi tạo dự án NodeJS: `npm init -y` (Giải thích ý nghĩa file `package.json`).
- Cài đặt ExpressJS: `npm install express` .
- Tạo file server chính (ví dụ: `server.js` hoặc `app.js`).
- Viết code server "Hello World" cơ bản:

```
// server.js

const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Hello World from Express!');
});

app.listen(port, () => {
  console.log(`Server listening at <http://localhost>:${port}`);
});
```

- Chạy server: `node server.js` .
- Kiểm tra bằng trình duyệt: Truy cập `http://localhost:3000` .
- 📌 **Lưu ý cho giảng viên:**
 - Nhấn mạnh sự tách biệt giữa Frontend và Backend.
 - Đảm bảo tất cả học viên chạy được server "Hello World". Khuyến khích sử dụng cổng khác 3000 nếu bị trùng.

Cải thiện Quy trình Phát triển & Giới thiệu RESTful API

1. Mục tiêu

- Cài đặt và sử dụng `nodemon` để tự động khởi động lại server khi có thay đổi code.
- Hiểu các khái niệm và nguyên tắc cơ bản của RESTful API.
- Nhận biết được cấu trúc của một request và response HTTP.

2. Nội dung

2.1 Nodemon

- Vấn đề: Phải khởi động lại server (`node server.js`) thủ công mỗi khi sửa code.
- Giải pháp: `nodemon` - theo dõi thay đổi file và tự động restart server.
- Cài đặt `nodemon` (dưới dạng dev dependency): `npm install --save-dev nodemon` .
- Cấu hình `scripts` trong `package.json` :

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

- Chạy server bằng nodemon: `npm run dev` .
- Thử thay đổi code và xem server tự khởi động lại.

2.2 Giới thiệu RESTful API

- **RESTful là gì?**
 - Hãy tưởng tượng bạn xây dựng một "kho chứa dữ liệu" trên mạng. RESTful là một **bộ quy tắc** để thiết kế "kho chứa dữ liệu" này sao cho khách hàng (các ứng dụng khác) dễ dàng “yêu cầu dữ liệu” và “nhận dữ liệu trả về”.
 - Thay vì nghĩ về nó như một thứ phức tạp, hãy xem RESTful như là **cách tốt nhất, phổ biến nhất để xây dựng API hiện nay**, giúp API dễ sử dụng, dễ bảo trì và dễ mở rộng.
- **Tại sao nên dùng RESTful? (Ưu điểm đơn giản)**
 - **Dễ hiểu và dễ dùng:** Giống như menu nhà hàng được viết rõ ràng, API RESTful được thiết kế để con người và máy móc đều dễ hiểu. URL rõ ràng, method HTTP trực quan.
 - **Hoạt động nhanh và hiệu quả:** RESTful giúp dữ liệu được truyền tải nhanh chóng và tiết kiệm tài nguyên, nhờ vào tính stateless và khả năng cache.
 - **Dễ dàng phát triển và nâng cấp:** Khi bạn muốn thêm món mới vào menu (thêm chức năng API), RESTful giúp việc này trở nên dễ dàng hơn, do tính module hóa và tuân thủ các chuẩn mực.

2.3 Các "quy tắc" chính của RESTful (chỉ cần nhớ 3 điều quan trọng nhất cho người mới bắt đầu)

- **1. "Tài nguyên" được gọi bằng "địa chỉ" rõ ràng (Resource Identification):**
 - Trong RESTful, mọi thứ bạn thao tác (dữ liệu, thông tin...) đều được xem là một "tài nguyên" (resource). Ví dụ: danh sách sản phẩm là một tài nguyên, thông tin của một người dùng cụ thể cũng là một tài nguyên.

- Để “yêu cầu” một tài nguyên, bạn cần biết “địa chỉ” của nó trên mạng, giống như địa chỉ nhà hàng. Trong API, “địa chỉ” này chính là **URL**. URL nên rõ ràng, dễ đoán và phản ánh tài nguyên.
- Ví dụ:
 - `http://myshop.com/products` (địa chỉ để lấy **danh sách** sản phẩm - resource: products - số nhiều)
 - `http://myshop.com/products/123` (địa chỉ để lấy **thông tin chi tiết** sản phẩm có ID là 123 - resource: product cụ thể - số ít)

• **2. "Hành động" được thể hiện bằng "động từ" rõ ràng (Self-descriptive Messages - HTTP Methods):**

- Khi bạn đến nơi “kho chứa tài nguyên” (server), bạn dùng “động từ” để nói bạn muốn làm gì: “cho tôi xem menu”, “tôi muốn đặt món này”, “hủy món vừa đặt”...
- Trong RESTful API, chúng ta cũng dùng “động từ” để nói với server chúng ta muốn làm gì với tài nguyên. Các “động từ” này chính là **HTTP Methods**: Mỗi method có ý nghĩa và mục đích rõ ràng.
 - **GET**: “Lấy thông tin” (ví dụ: xem menu, lấy danh sách sản phẩm) - **Đọc dữ liệu (Read)** - Sử dụng để truy xuất thông tin tài nguyên.
 - **POST**: “Gửi thông tin mới” (ví dụ: đặt món mới, tạo sản phẩm mới) - **Tạo dữ liệu (Create)** - Sử dụng để tạo mới tài nguyên.
 - **PUT/PATCH**: “Sửa thông tin đã có” (ví dụ: sửa thông tin món ăn, cập nhật sản phẩm) - **Cập nhật dữ liệu (Update)** - PUT thường thay thế toàn bộ tài nguyên, PATCH cập nhật một phần. Trong bài học cơ bản có thể dùng chung hoặc tập trung vào PATCH.
 - **DELETE**: “Xóa thông tin” (ví dụ: hủy món ăn, xóa sản phẩm) - **Xóa dữ liệu (Delete)** - Sử dụng để xóa tài nguyên.

• **3. "Tài nguyên" và "yêu cầu" được diễn tả bằng "ngôn ngữ chung" (Manipulation of Resources Through Representations):**

- Khi bạn yêu cầu “tài nguyên”, bạn và nhà hàng cần dùng chung một “ngôn ngữ” để hiểu nhau.
- Trong API, khi client và server trao đổi dữ liệu, họ cũng cần “ngôn ngữ chung”. “Ngôn ngữ” phổ biến nhất cho API ngày nay là **JSON** (JavaScript Object Notation). Nó giống như một cách viết dữ liệu theo cấu trúc rõ ràng, dễ đọc cho cả người và máy. API RESTful thường sử dụng JSON để gửi và nhận dữ liệu.
- Ví dụ dữ liệu JSON:

```
{
  "name": "Sản phẩm A",
  "price": 15000
}
```

• **Tóm lại:**

- RESTful API giống như một “kho chứa tài nguyên”, nơi bạn dùng “địa chỉ URL” để gọi “tài nguyên”
- Dùng “động từ HTTP Method” để thể hiện “hành động” bạn muốn làm với “tài nguyên” đó.
- Dùng “ngôn ngữ JSON” để “nói chuyện” với server, và nhận “tài nguyên” trả về.

• **Lưu ý cho giảng viên:**

- Giải thích ý nghĩa của từng HTTP method (GET, POST, PUT, DELETE) và ví dụ sử dụng trong thực tế, liên hệ với ví dụ “nhà hàng” nếu có thể. Ví dụ:
 - GET `/products` : “Xem menu sản phẩm”
 - POST `/products` : “Đặt món sản phẩm mới”
 - PUT/PATCH `/products/123` : “Sửa thông tin món sản phẩm số 123”
 - DELETE `/products/123` : “Hủy món sản phẩm số 123”
- Tập trung vào 3 “quy tắc” quan trọng đã đơn giản hóa của RESTful: **URL (địa chỉ), HTTP Methods (động từ), JSON (ngôn ngữ)**. Đây là nền tảng quan trọng nhất cho người mới bắt đầu.
- Không cần đi quá sâu vào các nguyên tắc khác của RESTful (ví dụ Stateless, Cacheable, Layered System, HATEOAS) ở giai đoạn này. Có thể giới thiệu sơ qua nếu có thời gian hoặc học viên có hứng thú, nhưng ưu tiên sự đơn giản và dễ hiểu.

- 🖱️ Khuyến khích học viên cài đặt công cụ kiểm thử API như **Postman** Hướng dẫn sơ qua cách sử dụng (tạo request, chọn method, gửi body, xem response).

Xây dựng API - Read Operations (GET)

1. Mục tiêu

- Tổ chức cấu trúc thư mục cơ bản (tùy chọn).
- Tạo dữ liệu tạm (mảng JavaScript).
- Xây dựng endpoint `GET` để lấy danh sách tất cả tài nguyên.
- Xây dựng endpoint `GET` để lấy một tài nguyên cụ thể bằng ID.
- Hiểu cách lấy tham số từ URL (`req.params`).

2. Nội dung

2.1 Chuẩn bị dữ liệu tạm

- Tạo một mảng các đối tượng JavaScript để mô phỏng database. Ví dụ: danh sách `todos` , `users` , `products` ...

```
// server.js

let todos = [
  { id: 1, title: 'Learn Express', completed: false },
  { id: 2, title: 'Build API', completed: false },
  { id: 3, title: 'Test API', completed: false }
];
let nextId = 4; // Để tạo ID cho item mới
```

- Đặt mảng này ở đầu file `server.js` (hoặc tách ra file riêng nếu muốn).

2.2 Endpoint: Lấy tất cả Todos (`GET /todos`)

- Định nghĩa route: `app.get('/todos', (req, res) => { ... });`
- Logic: Trả về toàn bộ mảng `todos` dưới dạng JSON.
- Sử dụng `res.json(data)` hoặc `res.status(200).json(data)` .

```
// server.js

app.get('/todos', (req, res) => {
  res.status(200).json(todos);
});
```

2.3 Endpoint: Lấy một Todo theo ID (`GET /todos/:id`)

- Định nghĩa route với tham số động: `app.get('/todos/:id', (req, res) => { ... });`
- **Giới thiệu `req.params`** : Cách lấy giá trị `id` từ URL (`req.params.id`). Lưu ý giá trị này thường là string, cần chuyển đổi sang số nếu ID là số (`parseInt()`).
- Logic:
 - Lấy `id` từ `req.params` .
 - Tìm `todo` trong mảng `todos` có `id` tương ứng.
 - Nếu tìm thấy: Trả về `todo` đó (`res.status(200).json(todo)`).
 - Nếu không tìm thấy: Trả về lỗi `404 Not Found` (`res.status(404).json({ message: 'Todo not found' })`).

```
// server.js
```

```
app.get('/todos/:id', (req, res) => {
  const todold = parseInt(req.params.id); // Chuyển sang số
  const todo = todos.find(t => t.id === todold);

  if (todo) {
    res.status(200).json(todo);
  } else {
    res.status(404).json({ message: `Todo with id ${todold} not found` });
  }
});
```

- 📌 Lưu ý cho giảng viên:
 - Giải thích rõ cách Express xử lý route và cách lấy `params`.
 - Nhấn mạnh tầm quan trọng của việc xử lý trường hợp không tìm thấy (404).
 - Hướng dẫn cách sử dụng `parseInt` và lý do tại sao cần chuyển đổi kiểu dữ liệu.
 - Khuyến khích cấu trúc code tốt hơn bằng cách tách route handlers ra các hàm riêng nếu có thời gian.

Xây dựng API - Create Operation (POST)

1. Mục tiêu

- Hiểu cách nhận dữ liệu từ client gửi lên trong request body.
- Sử dụng middleware `express.json()` để parse JSON body.
- Xây dựng endpoint `POST` để tạo mới một tài nguyên.

2. Nội dung

2.1 Nhận Request Body

- Giải thích: Khi client tạo mới (POST) hoặc cập nhật (PUT/PATCH), họ thường gửi dữ liệu trong phần body của request (thường là JSON).
- Express cần một "middleware" để đọc và phân tích (parse) body này trước khi route handler xử lý.

2.2 Middleware `express.json()`

- Middleware là các hàm chạy giữa request và response trong Express.
- `express.json()` là một built-in middleware để parse body có định dạng `application/json`.
- Cách sử dụng: `app.use(express.json());` - **Quan trọng:** Đặt dòng này ở đầu file, trước khi định nghĩa các routes cần dùng đến `req.body`.

2.3 Endpoint: Tạo mới Todo (`POST /todos`)

- Định nghĩa route: `app.post('/todos', (req, res) => { ... });`
- Giới thiệu `req.body`:** Sau khi dùng `express.json()`, dữ liệu JSON client gửi lên sẽ có trong `req.body`.
- Logic:
 - Lấy dữ liệu `todo` mới từ `req.body` (ví dụ: `{ title: 'New Task' }`).
 - (*Optional/Simple Validation*): Kiểm tra xem `req.body` có chứa các trường cần thiết không (ví dụ: `title`). Nếu không, trả về lỗi `400 Bad Request`.
 - Tạo `id` mới cho `todo` (sử dụng `nextId++` hoặc cách khác như `Date.now()` hoặc thư viện `uuid`).
 - Tạo đối tượng `todo` hoàn chỉnh (bao gồm `id`, `completed: false` mặc định,...).
 - Thêm `todo` mới vào mảng `todos`.
 - Trả về `todo` vừa tạo và status code `201 Created`.

```
// server.js

// Đặt ở đầu file, sau khi khởi tạo app
app.use(express.json()); // Middleware để parse JSON body

// ... các route GET ...

app.post('/todos', (req, res) => {
  const { title } = req.body; // Lấy title từ body

  // Validation đơn giản
  if (!title) {
    return res.status(400).json({ message: 'Title is required' });
  }

  const newTodo = {
    id: nextId++, // Tạo ID mới
    title: title,
    completed: false // Giá trị mặc định
  };

  todos.push(newTodo); // Thêm vào mảng

  res.status(201).json(newTodo); // Trả về todo vừa tạo
});
```

- 📌 **Lưu ý cho giảng viên:**
 - Giải thích **cực kỳ rõ** vai trò và vị trí đặt của middleware `express.json()` . Nhiều người mới bắt đầu hay quên hoặc đặt sai chỗ.
 - Nhấn mạnh việc tạo ID mới cho tài nguyên.
 - Giải thích tại sao nên trả về status `201 Created` và cả đối tượng vừa tạo.
 - Thảo luận về validation đơn giản và gợi ý rằng trong thực tế cần validation kỹ hơn.

Xây dựng API - Update Operation (PUT/PATCH)

1. Mục tiêu

- Xây dựng endpoint `PUT` để cập nhật toàn bộ một tài nguyên.
- (Tùy chọn) Giới thiệu sơ qua `PATCH` cho cập nhật một phần.
- Xử lý trường hợp tài nguyên cần cập nhật không tồn tại.

2. Nội dung

2.1 PUT vs PATCH (Nhắc lại)

- `PUT` : Thay thế toàn bộ tài nguyên bằng dữ liệu mới được gửi lên. Nếu thiếu trường nào trong body, trường đó có thể bị xóa hoặc đặt về mặc định (tùy logic xử lý).
- `PATCH` : Cập nhật một phần tài nguyên. Chỉ những trường được gửi lên trong body mới bị thay đổi.
- Trong bài này, tập trung vào `PUT` cho đơn giản.

2.2 Endpoint: Cập nhật Todo (`PUT /todos/:id`)

- Định nghĩa route: `app.put('/todos/:id', (req, res) => { ... });`
- Logic:
 - Lấy `id` từ `req.params` và dữ liệu cập nhật từ `req.body` .

- Tìm `index` của `todo` cần cập nhật trong mảng `todos` dựa vào `id`.
- Nếu không tìm thấy (`index === -1`): Trả về lỗi `404 Not Found`.
- Nếu tìm thấy:
 - *(Optional/Simple Validation)*: Kiểm tra dữ liệu trong `req.body`.
 - Tạo đối tượng `updatedTodo` mới, kết hợp `id` cũ với dữ liệu từ `req.body`. Đảm bảo giữ lại `id`.
 - Cập nhật `todo` tại `index` đó trong mảng `todos` bằng `updatedTodo`. `todos[index] = updatedTodo;`
 - Trả về `updatedTodo` và status `200 OK`.

```
// server.js

app.put('/todos/:id', (req, res) => {
  const todold = parseInt(req.params.id);
  const { title, completed } = req.body; // Lấy dữ liệu mới

  // Validation đơn giản
  if (title === undefined || completed === undefined) {
    return res.status(400).json({ message: 'Both title and completed status are required for PUT' });
  }
  if (typeof completed !== 'boolean') {
    return res.status(400).json({ message: 'Completed status must be a boolean' });
  }

  const todoIndex = todos.findIndex(t => t.id === todold);

  if (todoIndex === -1) {
    return res.status(404).json({ message: `Todo with id ${todold} not found` });
  }

  // Tạo object mới với id cũ và dữ liệu mới
  const updatedTodo = {
    id: todold, // Giữ lại ID cũ
    title: title,
    completed: completed
  };

  todos[todoIndex] = updatedTodo; // Thay thế todo cũ bằng todo mới

  res.status(200).json(updatedTodo); // Trả về todo đã cập nhật
});
```

- 📌 **Lưu ý cho giảng viên:**
 - Giải thích rõ logic tìm `index` và cập nhật phần tử trong mảng.
 - Nhấn mạnh sự khác biệt giữa `PUT` (thay thế toàn bộ) và cách triển khai nó (cần gửi đủ các trường). Nếu dùng `PATCH` thì logic sẽ khác (chỉ cập nhật các trường có trong `req.body`).
 - Nhắc lại việc xử lý lỗi 404.

Xây dựng API - Delete Operation (DELETE)

1. Mục tiêu

- Xây dựng endpoint `DELETE` để xóa một tài nguyên.
- Xử lý trường hợp tài nguyên cần xóa không tồn tại.
- Hiểu về status code `204 No Content`.

2. Nội dung

2.1 Endpoint: Xóa Todo (**DELETE /todos/:id**)

- Định nghĩa route: `app.delete('/todos/:id', (req, res) => { ... });`
- Logic:
 - Lấy `id` từ `req.params`.
 - Tìm `index` của `todo` cần xóa trong mảng `todos`.
 - Nếu không tìm thấy (`index === -1`): Trả về lỗi `404 Not Found`.
 - Nếu tìm thấy:
 - Sử dụng `todos.splice(index, 1)` để xóa phần tử tại `index` đó khỏi mảng.
 - Trả về status code `204 No Content`. Hoặc có thể trả về `200 OK` kèm message `{ message: 'Todo deleted successfully' }`. `204` thường được ưu tiên hơn cho DELETE thành công.

```
// server.js

app.delete('/todos/:id', (req, res) => {
  const todoid = parseInt(req.params.id);
  const todoIndex = todos.findIndex(t => t.id === todoid);

  if (todoIndex === -1) {
    return res.status(404).json({ message: `Todo with id ${todoid} not found` });
  }

  todos.splice(todoIndex, 1); // Xóa todo khỏi mảng

  // res.status(200).json({ message: 'Todo deleted successfully' });
  res.status(204).send(); // Trả về 204 No Content - không cần body
});
```

2.2 Hoạt động

- Học viên code endpoint DELETE.
- Sử dụng Postman/Insomnia:
 - Chọn method DELETE, URL là `/todos/:id` (thay `:id` bằng id của một todo đang tồn tại).
 - Gửi request và kiểm tra response (status 204, không có body).
 - Thử xóa một todo không tồn tại để xem lỗi 404.
 - Gửi request GET `/todos` lại để kiểm tra xem todo đã thực sự bị xóa.
- 📌 **Lưu ý cho giảng viên**
 - Giải thích cách hoạt động của `Array.prototype.splice()`.
 - Thảo luận về việc trả về `204 No Content` và tại sao nó phù hợp cho DELETE.
 - Đảm bảo học viên hiểu cách kiểm tra lại bằng GET sau khi DELETE.

📌 Code tham khảo sau khi hoàn thành `server.js`:

```
const express = require('express');
const app = express();
const port = 3000;

// Middleware để parse JSON body. **Quan trọng: Đặt trước các routes cần dùng req.body**
app.use(express.json());

// Dữ liệu tạm thời (mô phỏng database in-memory)
let todos = [
```



```

    { id: 1, title: 'Learn Express', completed: false },
    { id: 2, title: 'Build API', completed: false },
    { id: 3, title: 'Test API', completed: false }
  ];
  let nextId = 4; // Biến để tạo ID tự tăng cho todo mới

  // --- Endpoint GET: Lấy tất cả Todos ---
  app.get('/todos', (req, res) => {
    res.status(200).json(todos);
  });

  // --- Endpoint GET: Lấy một Todo theo ID ---
  app.get('/todos/:id', (req, res) => {
    const todold = parseInt(req.params.id); // Lấy ID từ params và chuyển sang số
    const todo = todos.find(t => t.id === todold); // Tìm todo trong mảng

    if (todo) {
      res.status(200).json(todo); // Trả về todo nếu tìm thấy
    } else {
      res.status(404).json({ message: `Todo with id ${todold} not found` }); // Lỗi 404 nếu không tìm thấy
    }
  });

  // --- Endpoint POST: Tạo mới Todo ---
  app.post('/todos', (req, res) => {
    const { title } = req.body; // Lấy title từ request body

    // Validation đơn giản: Kiểm tra title có được cung cấp không
    if (!title) {
      return res.status(400).json({ message: 'Title is required' }); // Lỗi 400 nếu thiếu title
    }

    const newTodo = {
      id: nextId++, // Tạo ID mới và tăng nextId
      title: title,
      completed: false // Giá trị mặc định cho completed
    };

    todos.push(newTodo); // Thêm todo mới vào mảng

    res.status(201).json(newTodo); // Trả về todo vừa tạo với status 201 (Created)
  });

  // --- Endpoint PUT: Cập nhật Todo ---
  app.put('/todos/:id', (req, res) => {
    const todold = parseInt(req.params.id); // Lấy ID từ params
    const { title, completed } = req.body; // Lấy title và completed từ body

    // Validation đơn giản cho PUT: Đảm bảo cả title và completed được cung cấp
    if (title === undefined || completed === undefined) {
      return res.status(400).json({ message: 'Both title and completed status are required for PUT' });
    }
    if (typeof completed !== 'boolean') {
      return res.status(400).json({ message: 'Completed status must be a boolean' });
    }

    const todoIndex = todos.findIndex(t => t.id === todold); // Tìm index của todo cần cập nhật

    if (todoIndex === -1) {
      return res.status(404).json({ message: `Todo with id ${todold} not found` }); // Lỗi 404 nếu không tìm thấy
    }
  });

```

```

}

const updatedTodo = {
  id: todold, // Giữ lại ID cũ
  title: title,
  completed: completed
};

todos[todoIndex] = updatedTodo; // Cập nhật todo trong mảng

res.status(200).json(updatedTodo); // Trả về todo đã cập nhật với status 200 (OK)
});

// --- Endpoint DELETE: Xóa Todo ---
app.delete('/todos/:id', (req, res) => {
  const todold = parseInt(req.params.id); // Lấy ID từ params
  const todoIndex = todos.findIndex(t => t.id === todold); // Tìm index của todo cần xóa

  if (todoIndex === -1) {
    return res.status(404).json({ message: `Todo with id ${todold} not found` }); // Lỗi 404 nếu không tìm thấy
  }

  todos.splice(todoIndex, 1); // Xóa todo khỏi mảng

  res.status(204).send(); // Trả về status 204 (No Content) cho thành công xóa, không có body
});

// Khởi động server
app.listen(port, () => {
  console.log(`Server listening at http://localhost:${port}`);
});

```